Creating a Simple wxWidgets Calculator Form Using wxFormBuilder

codvator

14-18 minutos

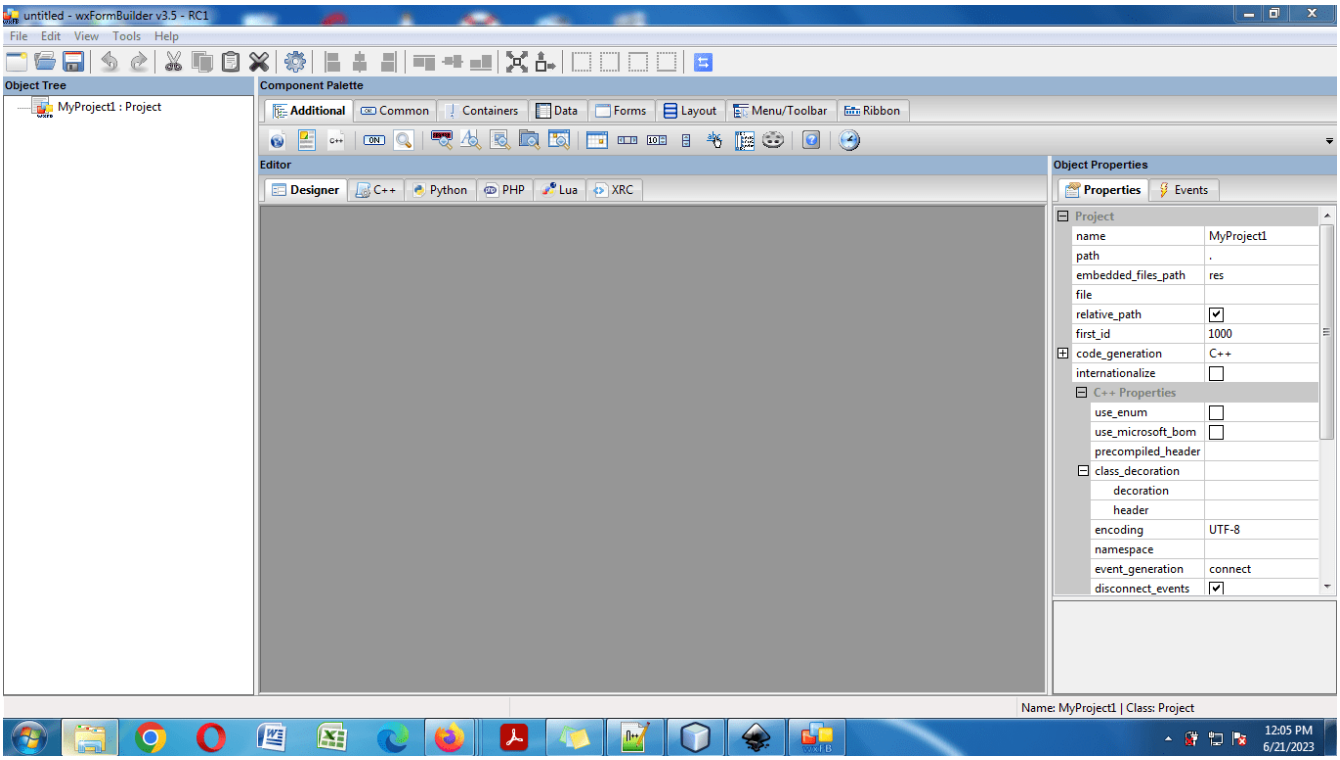
Posted on June 22, 2023 by codvator | Windows Development

Introduction

In this tutorial, we assume that the reader has prior knowledge about **C++** and the **wxWidgets API**, and how you can use it for cross platform application development. We shall focus on **wxFormBuilder** throughout this tutorial and how you can use it to create GUI forms for your desktop applications in the process creating a simple calculator form. In case, you need to refresh your knowledge about wxWidgets and to get started with it, check out this article. With this article read and with basic knowledge of C++ object oriented features, following through this tutorial won't really run you nuts. So let's begin.

So, perhaps your asking why all this fuss about wxFormBuilder. I mean I know how to write code and build my wxWidgets apps, so why do I really need wxFormBuilder. Okay, a short

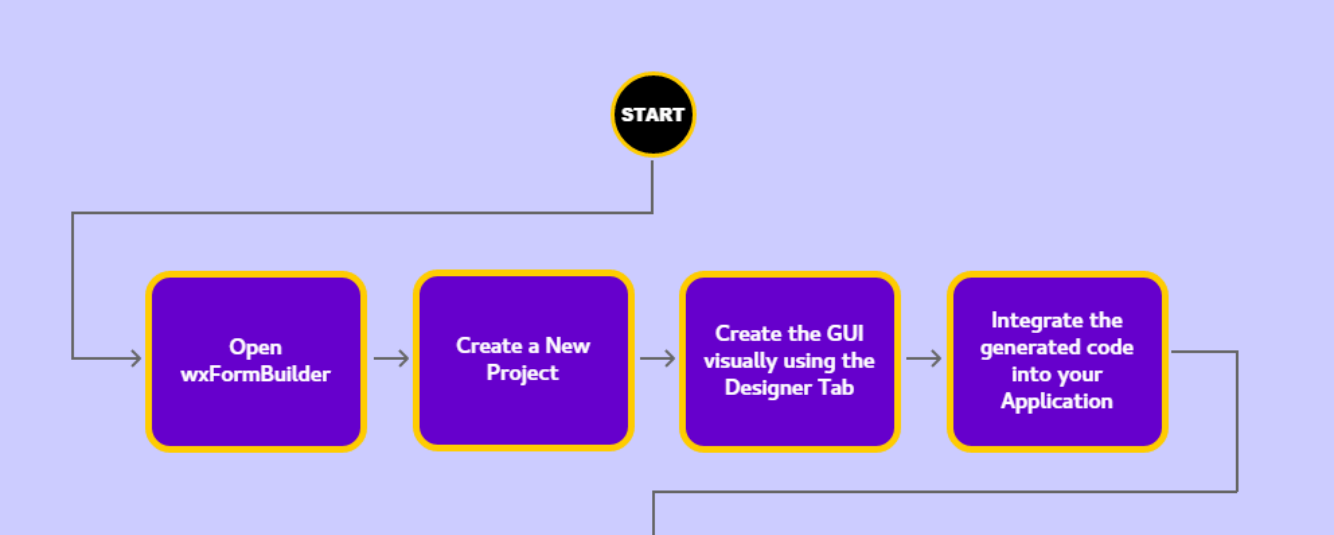
answer to your question will be that you need wxFormBuilder to ease your programming tasks and also reduce application development time. If you have written simple wxWidgets apps for some time, you will realize that its quite easy to build GUIs for these apps since there's generally little code to create the GUI. But as soon as you start working on larger projects and building larger applications, the GUIs associated with these apps also get more complicated and trust me, manually coding all the GUI will become pretty cumbersome. That is where the lovely, wxFormBuilder pops in, as the savior of the day. He will surely reduce any headaches that might come with writing mammoth GUI code.

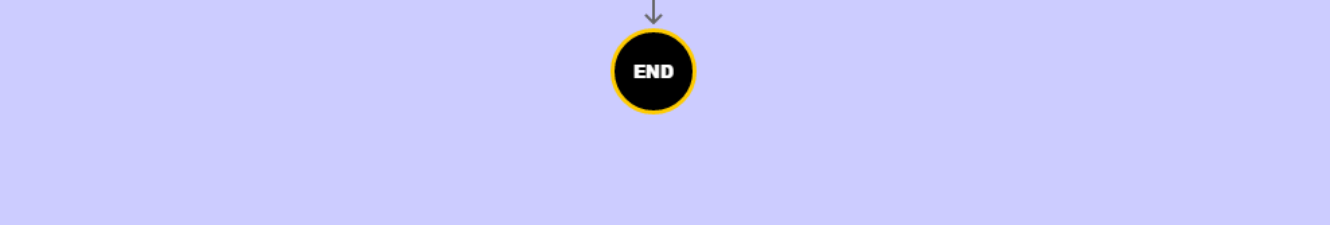


What is wxFormBuilder

wxFormBuilder is a basically a desktop application with a simple user interface (as seen from the photo above) that can help a developer automatically generate code for a particular wxWidgets application he or she is creating. The code generated by the application is GUI code with optional events attached to the widgets if needed. The application comes handy especially if one is creating complex GUIs.

The workflow of using wxFormBuilder is actually very simple assuming one has already downloaded the application onto his or her desktop. The steps are shown graphically below:

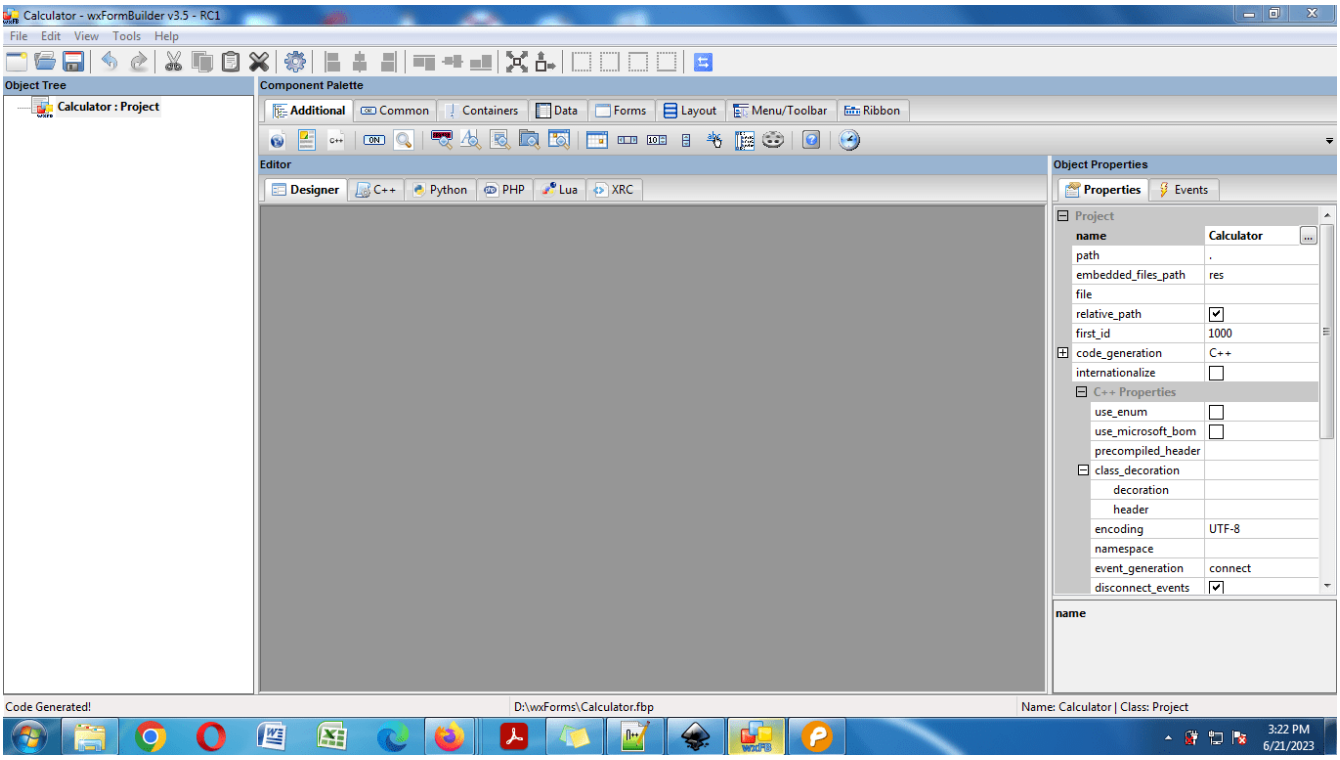




Creating a New wxFormBuilder Project

To get through the workflow above, let's create a simple wxWidgets application using wxFormBuilder. I assume you have already downloaded wxFormBuilder on your machine. To start, open the wxFormBuilder app from the windows start menu or double clicking its icon on the desktop. The main window of the application opens up.

What follows is creating a new project from **File** > **New Project**. Note that the main window usually opens with a new root project node already created for you in the **object tree** pane. The root node automatically created for you is named **MyProject1**. Let us save the project as **Calculator** by going to **File** > **Save As** and also change the name of the root project node in the **object properties** pane to **Calculator**. The project window will look as shown below.



As shown above, the main window has a **file menu**, a **tool bar**, three panes (**Object Tree**, **Editor**, **Object Properties**), and lastly a **Component Palette**. The Editor is where our visual GUI design work and code generation will take place. Notice that the editor has six tabs; **Designer**, **C++**, **Python**, **PHP**, **Lua** and lastly **XRC**. All design work appears in the **Designer** tab and for all components added into this tab, the corresponding code for various language ports is generated in the other tabs. The **Component Palette** acts as a repository where you can pick components and add them to the **Designer** tab. For any component added in the **Designer** Tab, the **Object Tree** Pane will be updated to reflect any changes. and it will hierarchically grow as you add more components to the Designer tab. The **Object Properties** pane is basically where you can change the properties of any widget or component added to the **Designer tab**. Such properties can be the **font**, **color**, **size**, **height**, **width** and many other properties. The properties also vary depending on what component is in focus.

Adding the Main GUI Application Window

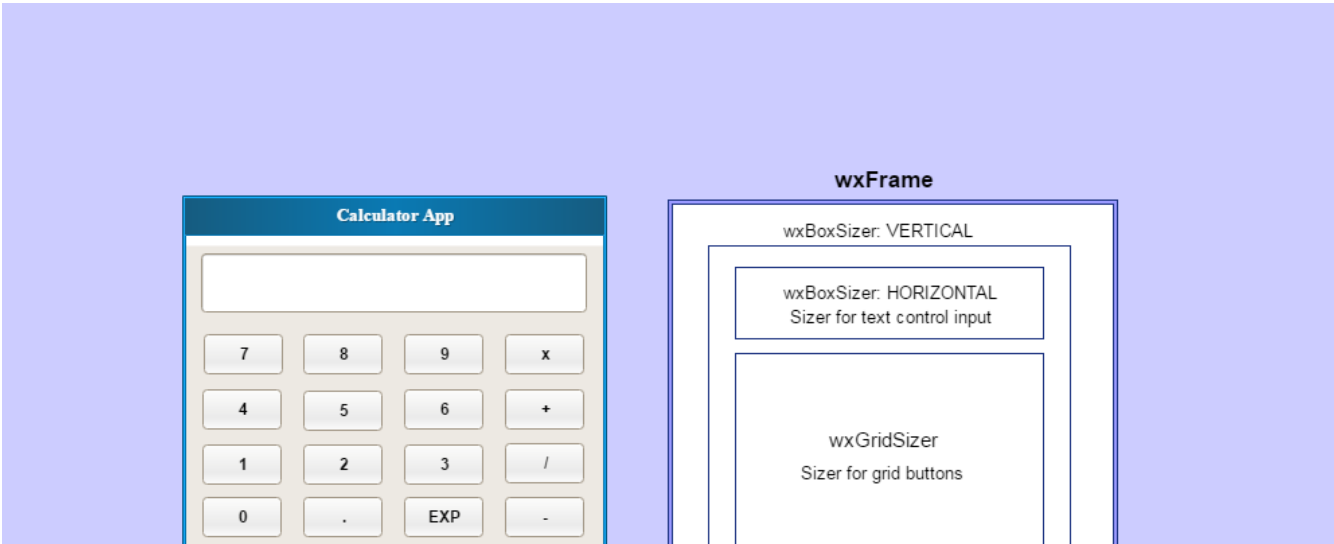
In wxWidgets, GUIs are hierarchical in nature in that at the top level, we have windows like **wxFrame**, **wxDialog** and **wxPopup** which act as top level windows for all other application windows. Other windows like **wxPanel**, **wxScrolledWindow**, **wxSplitterWindow**, **wxNotebook** etc act as containers for other windows like **wxButton**, **wxStaticText**, **wxTextCtrl** etc. The positioning of components is achieved using either absolute positioning or using Sizers. Sizers act as layout managers and are certainly the best way to manage and position components within containers.

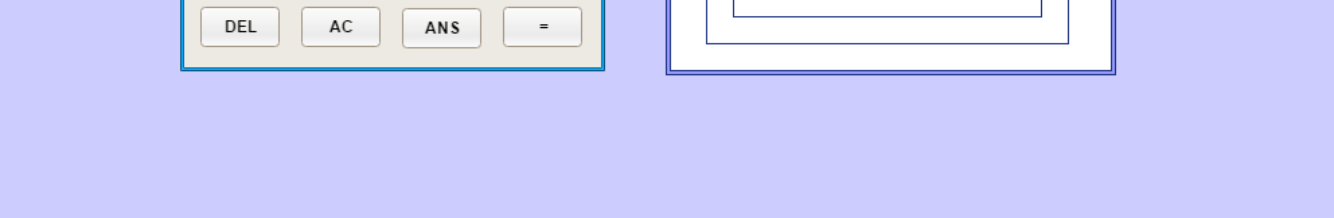
With that said, for our **Calculator** form, we shall add the main application window, a **wxFrame**.

1. Under **Component Palette**, click on the **Forms** tab to show all top level windows and select the **Frame** component by clicking on it. The **wxFrame** is added to the **Designer** tab and the **Object Tree** is also updated to show the window.
2. Check and affirm that the corresponding code is also generated in the language tabs. The code generated is used for porting to different language platforms for example an application being written in C++ will use code in the C++ tab, and so with other tabs. Please just make sure you don't edit or temper with any code within the tabs. Your focus should be on the **Designer** tab.
3. In case, you make any mistake, you can always undo any operation using **Ctrl + Z**. To save any changes made, use **Ctrl + S**
4. To change the properties of the **wxFrame**, use the **Object properties** pane. Change its **name** to **CalculatorFrame** and also change its **size** (**width**; **height**) to (**308;311**) Notice that you can still change other properties like **font**, **position**, etc.

Adding the Sizers

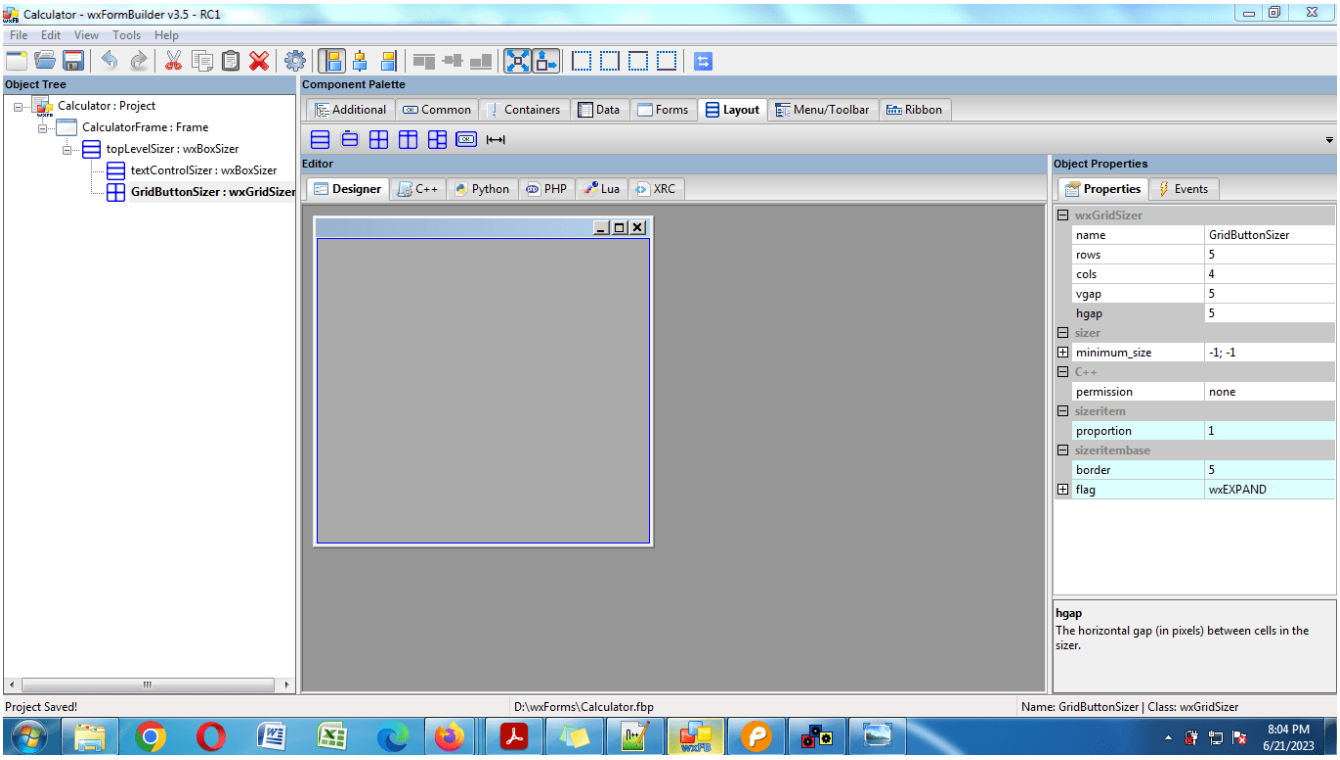
Should you try to add any component at this point, **wxFormBuilder** will display an error dialog, preventing you from adding the component. This is simply because you need to add a sizer before adding any component to manage layout in the application. We shall add three sizers; the top level parent sizer (a **wxBoxSizer** with **VERTICAL** orientation), and two other child sizers, a **wxBoxSizer** with **HORIZONTAL** orientation to manage the **wxTextCtrl** input and a **wxGridSizer** to manage the layout of grid buttons.





To add the above sizers,

1. Click on the **Layout** tab and select the **wxBoxSizer** to add it to the **wxFrame**.
2. In the **Object Properties**, change it's **name** to a fitting name you prefer like **topLevelSizer**. The **orient** property that defines its orientation is already **VERTICAL**, so no need to change it.
3. Now, with the **topLevelSizer** selected in the **Object Tree** pane, add another **wxBoxSizer**, and change its **name** to **textControlSizer** and its orientation to **HORIZONTAL**.
4. Also ensure that its **proportion** property is set to **0** To do this, simply click on the property value, a drop down will appear where you select the **HORIZONTAL** value.
5. Repeat the procedure above to add, another sizer, **wxGridSizer** below the **topLevelSizer**. Ensure that the **topLevelSizer** is selected before adding the **wxGridSizer** or else your sizer might be added under the **textControlSizer**.
6. Change the **name** of **wxGridSizer**, to **GridButtonSizer** and change the **rows**, **cols**, **vgap** and **hgap** properties to **5**, **4**, **5**, **5**. The properties **rows** and **cols** specify the number of grid rows and columns respectively while the **vgap** and **hgap** specify the vertical and horizontal gap (in pixels) between the components (buttons) to be added later. So far, our design will look like as shown below.



Adding the Text Control and Buttons

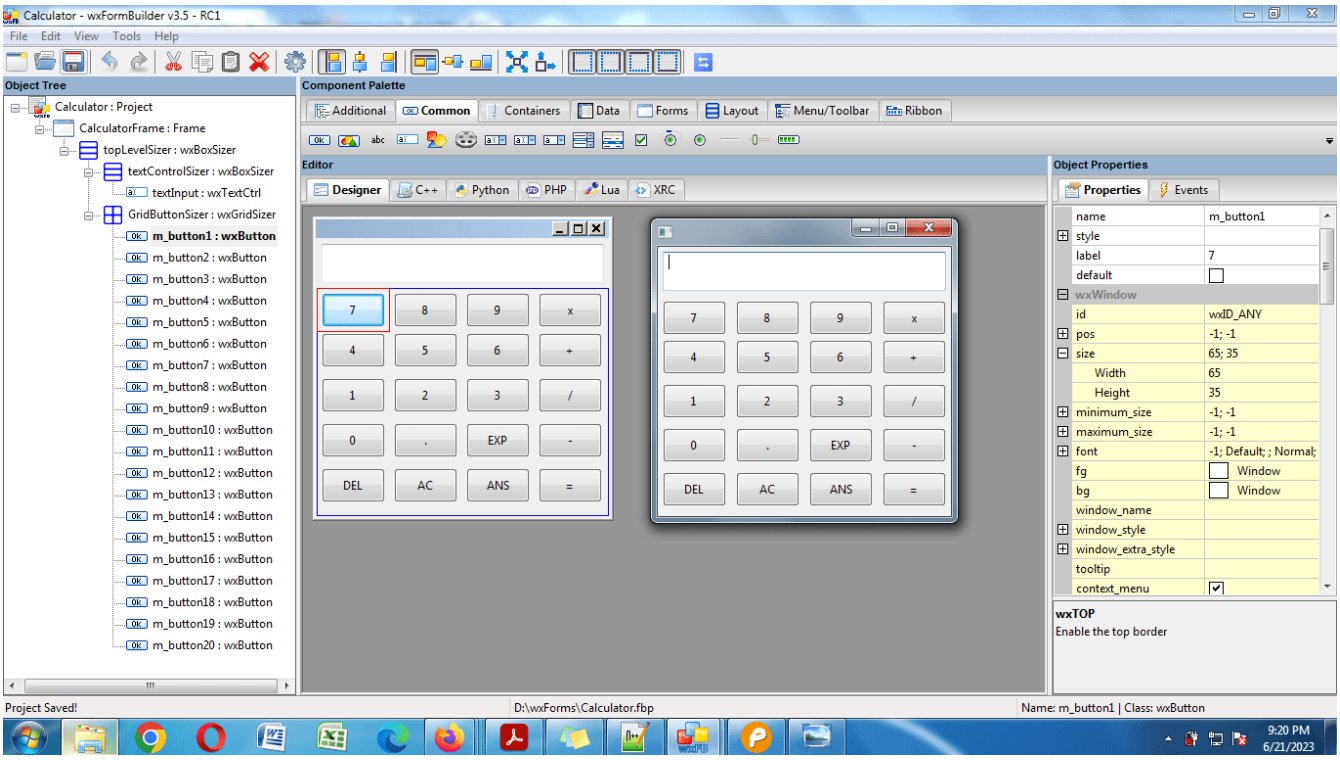
1. With the **textControlSizer** selected in the **Object Tree** pane, click on the **Common** tab and select the **wxTextCtrl** component to add it to the sizer. You realize that the text doesn't fill the whole length, so expand it horizontally using the **Expand** button in the tool bar at the top.

Also, change its **height** to **40** and its **name** to **textInput**.

2. Then select the **GridButtonSizer**, and add all the 20 buttons (**wxButton** components) into the sizer. The buttons are automatically arranged in a grid as you add each **wxButton**. For each button, set the label to its corresponding value like **7**, **8**, **9**, **4**, **5** etc and also set its **width** and **height** to **65** and **35** respectively.

3. Also remove the top and bottom borders of all other buttons except those in the first row. This is done by unchecking the **wxALL** flag and checking only the **wxLEFT** and **wxRIGHT** flags on the button. The **border** property must be set to **5** for all the buttons.

   After all this is done, our **Calculator** form will now be created. To see the actual look and feel of your Calculator GUI (the window on the right), go to the file menu **View** and click **XRC Window.** The window on the right appears. If you find that the spacing between buttons isn't appropriate, you can always make the necessary changes in the **Object Properties** pane.



To see all the generated code for your **Calculator** form, cycle through all the tabs and look through the code. Within the **C++** tab, wxFormBuilder generates the following code for the **header** and the **.cpp** file.

```
///////////////////////////////////////////////////////////////////////////
// C++ code generated with wxFormBuilder (version Jun 17 2015)
// http://www.wxformbuilder.org/
//
// PLEASE DO "NOT" EDIT THIS FILE!
///////////////////////////////////////////////////////////////////////////

#ifndef __NONAME_H__
#define __NONAME_H__

#include <wx/artprov.h>
```

```
#include <wx/xrc/xmlres.h>
#include <wx/string.h>
#include <wx/textctrl.h>
#include <wx/gdicmn.h>
#include <wx/font.h>
#include <wx/colour.h>
#include <wx/settings.h>
#include <wx/sizer.h>
#include <wx/button.h>
#include <wx/frame.h>

///////////////////////////////////////////////////////////////////////


///////////////////////////////////////////////////////////////////////
/// Class CalculatorFrame
///////////////////////////////////////////////////////////////////////
class CalculatorFrame : public wxFrame
{
        private:

        protected:
                wxTextCtrl* textInput;
                wxButton* m_button1;
                wxButton* m_button2;
                wxButton* m_button3;
                wxButton* m_button4;
                wxButton* m_button5;
                wxButton* m_button6;
                wxButton* m_button7;
                wxButton* m_button8;
                wxButton* m_button9;
                wxButton* m_button10;
                wxButton* m_button11;
                wxButton* m_button12;
                wxButton* m_button13;
                wxButton* m_button14;
                wxButton* m_button15;
                wxButton* m_button16;
                wxButton* m_button17;
                wxButton* m_button18;
                wxButton* m_button19;
                wxButton* m_button20;
```

```
	public:

		CalculatorFrame( wxWindow* parent, wxWindowID id = 
wxID_ANY, const wxString& title = wxEmptyString, const wxPoint& pos = 
wxDefaultPosition, const wxSize& size = wxSize( 308,311 ), long style 
= wxDEFAULT_FRAME_STYLE|wxTAB_TRAVERSAL );

		~CalculatorFrame();

};

#endif //__NONAME_H__
```

```
///////////////////////////////////////////////////////////////////////
// C++ code generated with wxFormBuilder (version Jun 17 2015)
// http://www.wxformbuilder.org/
//
// PLEASE DO "NOT" EDIT THIS FILE!
///////////////////////////////////////////////////////////////////////

#include "noname.h"

///////////////////////////////////////////////////////////////////////

CalculatorFrame::CalculatorFrame( wxWindow* parent, wxWindowID id, 
const wxString& title, const wxPoint& pos, const wxSize& size, long 
style ) : wxFrame( parent, id, title, pos, size, style )
{
	this->SetSizeHints( wxDefaultSize, wxDefaultSize );
	this->SetBackgroundColour( wxSystemSettings::GetColour( 
wxSYS_COLOUR_BTNFACE ) );

	wxBoxSizer* topLevelSizer;
	topLevelSizer = new wxBoxSizer( wxVERTICAL );

	wxBoxSizer* textControlSizer;
	textControlSizer = new wxBoxSizer( wxVERTICAL );

	textInput = new wxTextCtrl( this, wxID_ANY, wxEmptyString, 
wxDefaultPosition, wxSize( -1,40 ), 0 );
	textControlSizer->Add( textInput, 0, wxALL|wxEXPAND, 5 );

	topLevelSizer->Add( textControlSizer, 0, wxEXPAND, 5 );
```

```
	wxGridSizer* GridButtonSizer;
	GridButtonSizer = new wxGridSizer( 5, 4, 0, 0 );

	m_button1 = new wxButton( this, wxID_ANY, wxT("7"),
wxDefaultPosition, wxSize( 65,35 ), 0 );
	GridButtonSizer->Add( m_button1, 0, wxALL, 5 );

	m_button2 = new wxButton( this, wxID_ANY, wxT("8"),
wxDefaultPosition, wxSize( 65,35 ), 0 );
	GridButtonSizer->Add( m_button2, 0, wxALL, 5 );

	m_button3 = new wxButton( this, wxID_ANY, wxT("9"),
wxDefaultPosition, wxSize( 65,35 ), 0 );
	GridButtonSizer->Add( m_button3, 0, wxALL, 5 );

	m_button4 = new wxButton( this, wxID_ANY, wxT("x"),
wxDefaultPosition, wxSize( 65,35 ), 0 );
	GridButtonSizer->Add( m_button4, 0, wxALL, 5 );

	m_button5 = new wxButton( this, wxID_ANY, wxT("4"),
wxDefaultPosition, wxSize( 65,35 ), 0 );
	GridButtonSizer->Add( m_button5, 0, wxLEFT|wxRIGHT, 5 );

	m_button6 = new wxButton( this, wxID_ANY, wxT("5"),
wxDefaultPosition, wxSize( 65,35 ), 0 );
	GridButtonSizer->Add( m_button6, 0, wxLEFT|wxRIGHT, 5 );

	m_button7 = new wxButton( this, wxID_ANY, wxT("6"),
wxDefaultPosition, wxSize( 65,35 ), 0 );
	GridButtonSizer->Add( m_button7, 0, wxLEFT|wxRIGHT, 5 );

	m_button8 = new wxButton( this, wxID_ANY, wxT("+"),
wxDefaultPosition, wxSize( 65,35 ), 0 );
	GridButtonSizer->Add( m_button8, 0, wxLEFT|wxRIGHT, 5 );

	m_button9 = new wxButton( this, wxID_ANY, wxT("1"),
wxDefaultPosition, wxSize( 65,35 ), 0 );
	GridButtonSizer->Add( m_button9, 0, wxLEFT|wxRIGHT, 5 );

	m_button10 = new wxButton( this, wxID_ANY, wxT("2"),
wxDefaultPosition, wxSize( 65,35 ), 0 );
	GridButtonSizer->Add( m_button10, 0, wxLEFT|wxRIGHT, 5 );
```

```
	m_button11 = new wxButton( this, wxID_ANY, wxT("3"),
wxDefaultPosition, wxSize( 65,35 ), 0 );
	GridButtonSizer->Add( m_button11, 0, wxLEFT|wxRIGHT, 5 );

	m_button12 = new wxButton( this, wxID_ANY, wxT("/"),
wxDefaultPosition, wxSize( 65,35 ), 0 );
	GridButtonSizer->Add( m_button12, 0, wxLEFT|wxRIGHT, 5 );

	m_button13 = new wxButton( this, wxID_ANY, wxT("0"),
wxDefaultPosition, wxSize( 65,35 ), 0 );
	GridButtonSizer->Add( m_button13, 0, wxLEFT|wxRIGHT, 5 );

	m_button14 = new wxButton( this, wxID_ANY, wxT("."),
wxDefaultPosition, wxSize( 65,35 ), 0 );
	GridButtonSizer->Add( m_button14, 0, wxLEFT|wxRIGHT, 5 );

	m_button15 = new wxButton( this, wxID_ANY, wxT("EXP"),
wxDefaultPosition, wxSize( 65,35 ), 0 );
	GridButtonSizer->Add( m_button15, 0, wxLEFT|wxRIGHT, 5 );

	m_button16 = new wxButton( this, wxID_ANY, wxT("-"),
wxDefaultPosition, wxSize( 65,35 ), 0 );
	GridButtonSizer->Add( m_button16, 0, wxLEFT|wxRIGHT, 5 );

	m_button17 = new wxButton( this, wxID_ANY, wxT("DEL"),
wxDefaultPosition, wxSize( 65,35 ), 0 );
	GridButtonSizer->Add( m_button17, 0, wxLEFT|wxRIGHT, 5 );

	m_button18 = new wxButton( this, wxID_ANY, wxT("AC"),
wxDefaultPosition, wxSize( 65,35 ), 0 );
	GridButtonSizer->Add( m_button18, 0, wxLEFT|wxRIGHT, 5 );

	m_button19 = new wxButton( this, wxID_ANY, wxT("ANS"),
wxDefaultPosition, wxSize( 65,35 ), 0 );
	GridButtonSizer->Add( m_button19, 0, wxLEFT|wxRIGHT, 5 );

	m_button20 = new wxButton( this, wxID_ANY, wxT("="),
wxDefaultPosition, wxSize( 65,35 ), 0 );
	GridButtonSizer->Add( m_button20, 0, wxLEFT|wxRIGHT, 5 );


	topLevelSizer->Add( GridButtonSizer, 1, wxEXPAND, 5 );
```

```
        this->SetSizer( topLevelSizer );
        this->Layout();

        this->Centre( wxBOTH );
}

CalculatorFrame::~CalculatorFrame()
{
}
```

The code above can then be copied and pasted into an IDE like MS Visual Studio to integrate it into your code if you were developing a calculator application. The code that creates the GUI is that one within the constructor CalculatorFrame::CalculatorFrame()